

LAB GUIDE WITH SOLUTIONS

Redis Persistence Labs

Six hands-on labs in Docker, with solutions inline

Module 07 · NoSQL & Redis

ICICI Bank Learning Academy

Enterprise Data Engineering with SQL, Redis & Kafka

Module 07 · NoSQL & Redis · Instructor-led · For classroom use

How to use this lab guide

Six labs, sized for a three-hour hands-on session on Redis persistence - RDB, AOF, the Redis 7 hybrid, and what changes when the server runs inside a container. Every lab states its objective, shows the exact commands, and shows the output you should see. Solutions are inline - this document is the instructor copy and the participant copy at the same time.

All commands are **Docker-native** - there is no WSL2 or a locally installed redis-server anywhere in this guide. Every command either runs on your host against the Docker CLI, or is prefixed with `docker exec <container>` to run inside the container. Windows, macOS and Linux all use the same commands unchanged.

Lab	Title	Covers	Time
P.1	RDB in a named volume - and the pitfall without one	Deck 6 §1, §4	35 min
P.2	Enabling AOF - Multi Part AOF, live	Deck 6 §2	25 min
P.3	AOF rewrite - BGREWRITEAOF, measured	Deck 6 §2	20 min
P.4	Breaking AOF on purpose, and repairing it	Deck 6 §2	25 min
P.5	docker-compose: healthcheck, down vs down -v	Deck 6 §4	30 min
P.6	Backup and the disaster-recovery restore drill	Deck 6 §4	25 min

WATCH OUT Every key created in these labs starts with `icici:`. Every container is named so it is obvious which lab it belongs to, and every lab tells you exactly what to remove at the end - nothing is left running by accident.

READ THIS Conventions: lines starting with `$` run in your own shell (PowerShell, bash, whichever you use). Lines with no prefix run inside `redis-cli`. Indented lines in teal are the output you should see - they were captured from a real run, not written by hand.

LAB P.1 · 35 MINUTES

RDB in a Named Volume - and the Pitfall Without One

Objective	Prerequisite
Prove that a snapshot survives a container being recreated - and reproduce, on purpose, the one mistake that loses everything.	Docker Desktop running. No prior lab needed.

Part A · The pitfall, first - so you know what you are avoiding

Start a container with no volume at all, write one key, and remove it. This is deliberately the wrong way to do it.

```
$ docker run -d --name r -p 6398:6379 redis:7-alpine
$ docker exec r redis-cli SET icici:demo "no volume"
OK
$ docker exec r redis-cli DBSIZE
(integer) 1

$ docker rm -f r
$ docker run -d --name r -p 6398:6379 redis:7-alpine
$ docker exec r redis-cli DBSIZE
(integer) 0 # measured - the key is gone
$ docker exec r redis-cli GET icici:demo
(nil)
$ docker rm -f r
```

IMPORTANT This has nothing to do with whether RDB or AOF was configured. A container with no volume writes its files into its own writable layer, and `docker rm` deletes that layer along with the container.

Part B · The fix - a named volume

```
$ docker volume create icici_redis_data
$ docker run -d --name icici-redis -p 6379:6379 \
  -v icici_redis_data:/data redis:7-alpine \
  redis-server --save 900 1 --save 300 10 --save 60 10000 --dir /data
$ docker exec icici-redis redis-cli PING
PONG
```

Step 1 · Write data and force a snapshot

```
$ docker exec icici-redis redis-cli MSET \
  icici:bal:9003 341244.80 icici:bal:7711 89210.50
OK
$ docker exec icici-redis redis-cli SET icici:persist:test "alive before restart"
OK
$ docker exec icici-redis redis-cli DBSIZE
(integer) 3
$ docker exec icici-redis redis-cli BGSAVE
Background saving started
$ docker exec icici-redis redis-cli INFO persistence | grep rdb_last
rdb_last_save_time:1790121734
rdb_last_bgsave_status:ok
```

Step 2 · Look at the file that was written

```
$ docker exec icici-redis ls -la /data
-rw----- 1 redis redis 186 Sep 23 00:02 dump.rdb
```

Step 3 · Recreate the container from the SAME volume

This is the moment that matters - stop the container entirely, throw it away, and start a brand new one, pointed at the same named volume.

```
$ docker rm -f icici-redis
$ docker run -d --name icici-redis -p 6379:6379 \
  -v icici_redis_data:/data redis:7-alpine \
  redis-server --save 900 1 --save 300 10 --save 60 10000 --dir /data
$ docker exec icici-redis redis-cli DBSIZE
(integer) 3                                     # survived
$ docker exec icici-redis redis-cli GET icici:persist:test
"alive before restart"
$ docker exec icici-redis redis-cli GET icici:bal:9003
"341244.80"
```

Solution · Why Part A lost data and Part B did not

Identical Redis config in both cases. The only difference is `-v icici_redis_data:/data`. A named volume is a storage object Docker manages independently of any one container; a container references it, but removing the container does not remove the volume. Without that flag, `dump.rdb` was written to a location that existed only as long as the container did.

CHECKPOINT You have reproduced total data loss on purpose, and fixed it with one flag. Keep `icici-redis` and `icici_redis_data` running - the next labs build on them.

LAB P.2 · 25 MINUTES

Enabling AOF - Multi Part AOF, Live

Objective	Prerequisite
Turn on the append-only log against a running container and inspect the three files Redis 7 splits it into.	Lab P.1 complete - icici-redis still running.

Step 1 · Enable it at runtime

```
$ docker exec icici-redis redis-cli CONFIG SET appendonly yes
OK
$ docker exec icici-redis redis-cli CONFIG GET appendonly
1) "appendonly" 2) "yes"
```

WATCH OUT CONFIG SET is live but not persisted. If the container restarts with the old command line (no --appendonly yes), AOF is off again. Lab P.5's docker-compose setup bakes it into the config file instead, which is the production way to do this.

Step 2 · Look at what appeared on disk

```
$ docker exec icici-redis ls -la /data
drwx----- 2 redis redis 4096 appendonlydir
-rw----- 1 redis redis 186 dump.rdb
$ docker exec icici-redis ls -la /data/appendonlydir
-rw----- 1 redis redis 186 appendonly.aof.1.base.rdb
-rw----- 1 redis redis 0 appendonly.aof.1.incr.aof
-rw----- 1 redis redis 88 appendonly.aof.manifest
```

Step 3 · Read the manifest - it is the index

```
$ docker exec icici-redis cat /data/appendonlydir/appendonly.aof.manifest
file appendonly.aof.1.base.rdb seq 1 type b
file appendonly.aof.1.incr.aof seq 1 type i
```

type **b** is the base file - the starting snapshot, written in RDB format. type **i** is the incremental file - every write command since. Redis 7 loads the base first, then replays the incr file on top of it.

Step 4 · Confirm the fsync policy

```
$ docker exec icici-redis redis-cli CONFIG GET appendfsync
1) "appendfsync" 2) "everysec"
```

Solution · Why three files instead of one

Before Redis 7, AOF was a single file that only ever grew, and a rewrite had to be done very carefully to avoid corrupting the one file the server needed on the next start. Splitting base from incr means a rewrite can write a brand-new base file, and only delete the old one after the new one and the manifest are safely on disk - there is no moment where the working set is a half-written file.

CHECKPOINT You can see, by file size and by reading the manifest, exactly what Multi Part AOF looks like on disk - not just what the docs say it does.

LAB P.3 · 20 MINUTES

AOF Rewrite - BGREWRITEAOF, Measured

Objective	Prerequisite
Grow the AOF tail, trigger a rewrite, and measure the file sizes before and after in the same run.	Lab P.2 complete.

Step 1 · Grow the incr file

```
$ for i in $(seq 1 50); do docker exec icici-redis redis-cli \
  SET icici:cnt:txn:$i $((i*1000)) > /dev/null; done
$ docker exec icici-redis redis-cli DBSIZE
(integer) 53
$ docker exec icici-redis ls -la /data/appendonlydir
-rw----- 1 redis redis 2355 appendonly.aof.1.incr.aof
```

Step 2 · Rewrite it

```
$ docker exec icici-redis redis-cli BGREWRITEAOF
Background append only file rewriting started
$ docker exec icici-redis ls -la /data/appendonlydir
-rw----- 1 redis redis 1263 appendonly.aof.2.base.rdb
-rw----- 1 redis redis 0 appendonly.aof.2.incr.aof
-rw----- 1 redis redis 88 appendonly.aof.manifest
$ docker exec icici-redis cat /data/appendonlydir/appendonly.aof.manifest
file appendonly.aof.2.base.rdb seq 2 type b
file appendonly.aof.2.incr.aof seq 2 type i
```

Step 3 · Read the persistence stats

```
$ docker exec icici-redis redis-cli INFO persistence | grep aof_
aof_enabled:1
aof_rewrite_in_progress:0
aof_last_bgrewrite_status:ok
aof_current_size:1263
aof_base_size:1263
```

Solution · Read the numbers

Before	After	What happened
incr.aof: 2,355 bytes (53 SET commands as text)	base.rdb: 1,263 bytes (53 keys, RDB format)	The rewrite replaced 53 individual commands with one compact snapshot
seq 1	seq 2	A new sequence number - old files are never edited, only replaced
(no incr for this seq yet)	incr.aof: 0 bytes	Fresh incr file, ready for the next writes

CHECKPOINT auto-aof-rewrite-percentage 100 with auto-aof-rewrite-min-size 64mb means this happens automatically once the file has doubled past 64 MB - you just did by hand what Redis does for itself in production.

LAB P.4 · 25 MINUTES

Breaking AOF on Purpose, and Repairing It

Objective	Prerequisite
Corrupt the incremental file, watch the server refuse to start, and fix it with redis-check-aof.	Lab P.3 complete. This lab stops and restarts ici-ci-redis - expected.

Step 1 · Stop the server and corrupt the incr file

You cannot safely edit a file the server has open, so stop the container first.

```
$ docker stop icici-redis  
$ docker run --rm -v icici_redis_data:/data redis:7-alpine sh -c \  
    "printf '\n\n*3\r\n\n$3\r\n\r\nSET\r\n\r\nncorrupt-partial-write' \  
    >> /data/appendonlydir/appendonly.aof.2.incr.aof"
```

Step 2 · Try to start a server against the corrupted data

```
$ docker run --rm -v icici_redis_data:/data redis:7-alpine \
redis-server --dir /data --appendonly yes
... Reading RDB base file on AOF loading...
... DB loaded from base file appendonly.aof.2.base.rdb: 53 keys
# Bad file format reading the append only file
# appendonly.aof.2.incr.aof: make a backup of your AOF file,
# then use ./redis-check-aof --fix <filename.manifest>
```

The server loaded the base file (53 keys, unaffected) and then refused outright at the corrupted incr file. It does not guess, truncate, or skip - it stops and tells you exactly which tool to run.

Step 3 · Repair it

```
$ docker run --rm -v icici_redis_data:/data redis:7-alpine sh -c \
    "echo y | redis-check-aof --fix \
    /data/appendonlydir/appendonly.aof.manifest"
Start checking Multi Part AOF
BASE AOF appendonly.aof.2.base.rdb is valid
Start to check INCR files.
AOF appendonly.aof.2.incr.aof format error
This will shrink the AOF appendonly.aof.2.incr.aof
from 35 bytes, with 35 bytes, to 0 bytes
Continue? [y/N]: Successfully truncated AOF appendonly.aof.2.incr.aof
All AOF files and manifest are valid
```

Step 4 · Confirm it starts clean

```
$ docker run --rm -v icici_redis_data:/data redis:7-alpine \
    timeout 3 redis-server --dir /data --appendonly yes
... Opening AOF incr file appendonly.aof.2.incr.aof on server start
... Ready to accept connections tcp
```

Step 5 · Bring the real container back

```
$ docker start icici-redis
```

```
$ docker exec icici-redis redis-cli DBSIZE  
(integer) 53 # nothing was lost
```

Solution · What redis-check-aof actually did

It told you the exact byte count of the corruption (35 bytes - precisely what this lab appended by hand), asked before changing anything, and truncated only the damaged tail. The base file and every key it held (53) were never touched. This is the difference between an AOF being merely "broken" and an AOF being unrecoverable - a corrupted tail costs you, at worst, the writes since the last good fsync, never the whole dataset.

CHECKPOINT You have watched Redis refuse to start on bad data rather than silently load a partial state, and you know the exact command to recover from it.

LAB P.5 · 30 MINUTES

docker-compose: Healthcheck, down vs down -v

Objective	Prerequisite
Run the training setup the way it is meant to be run, and prove the one command that destroys data on purpose.	Lab P.4 complete. Uses persistence_docker/docker-compose.yml from this pack.

Step 1 · Clean up the manual container from earlier labs

```
$ docker rm -f icici-redis
$ docker volume rm icici_redis_data
```

Step 2 · Bring the stack up

```
$ cd persistence_docker
$ cp .env.example .env
# edit .env and set a real REDIS_PASSWORD
$ docker compose up -d
$ docker compose ps
NAME                STATUS
icici-redis         Up 17 seconds (healthy)
```

STATUS reaching healthy means the healthcheck's own redis-cli ... PING succeeded - not just that the process started.

Step 3 · Write a key, then a routine restart

```
$ source .env
$ docker exec icici-redis redis-cli -a "$REDIS_PASSWORD" --no-auth-warning \
  SET icici:compose:test "up via compose"
OK

$ docker compose down
$ docker volume ls | grep icici
icici_redis_data      <- still there

$ docker compose up -d
$ docker exec icici-redis redis-cli -a "$REDIS_PASSWORD" --no-auth-warning \
  GET icici:compose:test
"up via compose"      <- survived
```

Step 4 · The command that does NOT keep your data

```
$ docker compose down -v
Volume icici_redis_data Removing
Volume icici_redis_data Removed
$ docker volume ls | grep icici
(nothing)              <- gone for good
```

IMPORTANT This is not a bug in compose - -v means exactly "and the volumes". The mistake is never in the flag; it is in running it without checking which environment your terminal is pointed at.

Solution · The rule for this command in real environments

Command	Removes	Use it when
<code>docker compose stop</code>	Nothing - containers just stop	You want to pause, not clean up
<code>docker compose down</code>	Containers and the default network	A routine restart or redeploy
<code>docker compose down -v</code>	The above, plus every named volume	You have already backed up and want a clean slate

CHECKPOINT You have run the training stack the production way, and you have a muscle-memory reason to pause before ever typing `-v`.

LAB P.6 · 25 MINUTES

Backup and the Disaster-Recovery Restore Drill

Objective	Prerequisite
Copy the data out of a running container, and prove it can rebuild a completely fresh one from nothing but that file.	Lab P.5 complete - icici-redis running via compose.

Step 1 · Force a snapshot and copy it out

```
$ source .env
$ docker exec icici-redis redis-cli -a "$REDIS_PASSWORD" --no-auth-warning BGSAVE
Background saving started
$ docker cp icici-redis:/data/. ./backup/
$ ls backup/
dump.rdb appendonlydir/
```

Step 2 · Build a completely fresh container and volume

Not the same volume - a brand new one, as if this were a different machine entirely.

```
$ docker volume create icici_redis_restored
$ docker run -d --name icici-redis-restored \
  -v icici_redis_restored:/data redis:7-alpine \
  redis-server --dir /data
$ docker stop icici-redis-restored
$ docker cp backup/dump.rdb icici-redis-restored:/data/dump.rdb
$ docker start icici-redis-restored
```

Step 3 · Verify - this is the step that actually proves anything

```
$ docker exec icici-redis-restored redis-cli DBSIZE
(integer) 53 # every key came back
$ docker exec icici-redis-restored redis-cli GET icici:persist:test
"alive before restart"
$ docker exec icici-redis-restored redis-cli GET icici:bal:9003
"341244.80" # measured, same run
```

Step 4 · Validate the backup file itself

```
$ docker run --rm -v icici_redis_restored:/data redis:7-alpine \
  redis-check-rdb /data/dump.rdb
[offset 186] Checksum OK
[offset 186] \o/ RDB looks OK! \o/
```

Solution · Why this drill, and not just "BGSAVE works"

A backup job that runs every night and never gets tested is a belief, not a guarantee. The only way to know a restore actually works is to run it against infrastructure that has never seen your data before - a fresh volume, a fresh container - and check the keys by hand. Everything up to Step 2 could succeed even if the restore procedure itself were broken; Step 3 is the only step that proves anything.

Cleaning up

```
$ docker rm -f icici-redis-restored
$ docker volume rm icici_redis_restored
$ docker compose down -v          # only if you are done with the whole lab set
$ rm -rf backup .env
```

CHECKPOINT You have a backup you have proven restores, on infrastructure that never saw the original container - which is the only kind of backup worth trusting.

What you should be able to do now

- Explain what BGSAVE actually does (fork, copy-on-write, atomic rename) and prove a snapshot survives a container being recreated - and reproduce data loss without a volume.
- Enable AOF at runtime and read Redis 7's Multi Part AOF layout: base.rdb, incr.aof, manifest.
- Trigger and measure a BGREWRITEAOF, and explain why the sequence number changes.
- Corrupt an AOF file, read the server's exact refusal, and repair it with redis-check-aof --fix.
- Run the training stack with docker-compose, reach a healthy status, and state precisely what docker compose down -v deletes that plain down does not.
- Back up a running container's data and restore it into infrastructure that never saw the original - and know why that is the only test that counts.

NEXT Next: open 06_Redis_Persistence.pptx and re-run any command on this page that surprised you - everything in that deck was captured from the same kind of session as this guide.